ChatGPT

Local vs Global Optimization and the Basin Hopping Algorithm

Introduction: Optimization problems often have **multiple optima** – some are *local optima* and one (or more) may be the *global optimum*. New data scientists frequently run into the **"local minima trap"**, where a local optimization method converges to a suboptimal solution that is not the true global minimum. In this comprehensive overview, we will clarify the difference between local and global optimization, discuss why local methods can get "stuck," and explore how to overcome this using global strategies. In particular, we will dive into the **Basin Hopping** algorithm – a powerful global optimization technique that can be applied on top of any local optimizer (even one you've developed yourself). We'll also include visual examples, comparisons, and some mathematical formulations to solidify these concepts.

Local vs. Global Optimization

Definition of Local vs Global Minimum: A **local minimum** (or local optimum) of a function is a point where the function value is smaller than at any nearby points, but not necessarily the smallest overall. Formally, a point x^* is a local minimum of $f(x)$ if there exists some neighborhood around x^* such that $f(x^*) \leq f(x)$ for all x in that neighborhood [1] [2]. In contrast, a **global minimum** is a point where $f(x^*) \leq f(x)$ for *all* feasible x in the domain – i.e. it's the absolute lowest value of the function [1] [2]. In simple terms, the global minimum is the very deepest valley of the entire landscape, whereas a local minimum is just a smaller valley that might be higher than the deepest one.

- *Example:* Imagine a hilly terrain where the height at each point is given by some function $f(x)$. A **global minimum** is the lowest valley in the whole region. A **local minimum** is any other valley that is lower than its immediate surroundings but higher than the lowest valley. There can be many local minima but only one global minimum (for a given connected domain; there could be multiple global minima if the lowest value occurs at several points).

- **Convex vs Non-Convex Problems:** In *convex* optimization (where $f(x)$ is a convex function on a convex domain), any local minimum is also a global minimum [3]. This is why convex problems are "easy" – a simple local search will find the global optimum. However, for *non-convex* or **multimodal** functions (having multiple peaks and valleys), local minima need not be global. These problems can fool local algorithms into stopping at suboptimal points. Most real-world machine learning loss surfaces and complex optimization problems are multimodal and non-convex.

Why Local Methods Can Get Stuck: Local optimization algorithms (such as gradient descent, Newton's method, or standard simplex search) typically **converge to the nearest optima "in the basin of attraction" of the starting point** [4]. The *basin of attraction* for a given minimum is the region of the input space that will lead the algorithm to that minimum. If you start a local search in a particular basin, the algorithm will descend to the bottom of that basin – which could be a local minimum, not the global one. Solvers like gradient descent *only guarantee reaching a local minimum*, and *there's no guarantee that the local*

minimum found is global [5]. In fact, gradient-based methods can **get stuck** at a plateau or poor local valley, especially if the landscape has many such traps.

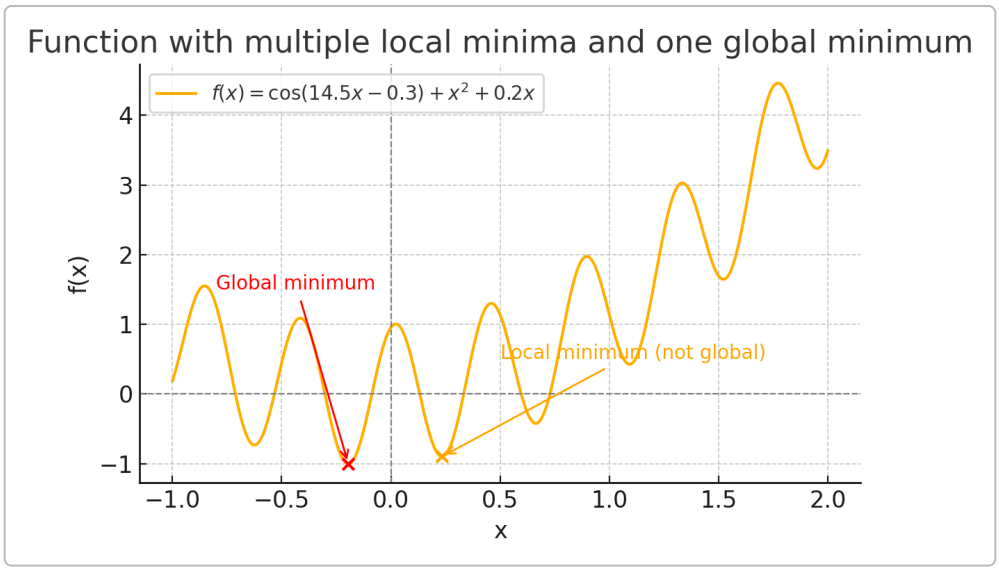

Figure: A multimodal function with multiple local minima and one global minimum. The lowest valley (red X) is the global minimum, while another nearby valley (orange X) is a higher local minimum. A local optimization starting near the orange point would get "trapped" there, since it's the lowest point in that vicinity, even though a much lower (better) optimum exists elsewhere.

As an illustrative example, consider the 1-D function plotted above:

$$f(x) = \cos(14.5\,x - 0.3) + x^2 + 0.2x,$$

which has a wavy "rugged" shape. You can see several dips (local minima). A naive local search starting at, say, $x = 1.0$ would roll down to the nearest valley (around $x \approx 0.66$ in orange) and stop, *missing* the deeper valley at $x \approx -0.2$ (in red) which is the true global minimum. This highlights the risk of using only local optimization: **you might only find a "good-enough" solution (locally optimal) instead of the best possible solution (globally optimal)** [4].

Real-World Impact: For new data scientists training machine learning models, this issue often arises when optimizing a non-convex loss function. The training process (e.g. gradient descent on a neural network loss) might converge to some set of parameters that yields low error, but perhaps not the lowest possible error. If the algorithm got caught in a local minimum, a better model configuration could exist. Recognizing this possibility is important – otherwise one might mistakenly believe the solution is optimal when it's not.

How to Avoid Local Minima Traps: To find a global optimum (or at least get closer to it), one typically needs **global optimization techniques** or strategies that explore multiple basins:

- **Multiple Random Restarts:** One simple approach is to run a local optimizer multiple times with different random initializations and take the best result [6]. This "multi-start" approach increases the

chance that at least one run finds the basin of the global minimum. (In fact, MATLAB's GlobalSearch and MultiStart solvers automate this kind of random multi-start search [7].)

- **Heuristic/Metaheuristic Algorithms:** Techniques like **simulated annealing**, **evolutionary/genetic algorithms**, **particle swarm optimization**, etc., are designed to *explore the search space globally* and can escape local minima by allowing occasional uphill moves or population-based search. For example, simulated annealing starts like a random walk that occasionally accepts worse solutions and gradually becomes more selective, in analogy to cooling metal to find low-energy states. These methods have mechanisms to jump out of local basins and explore broadly.

- **Regularization and Smoothness:** In machine learning, sometimes modifying the problem can help – e.g. adding a regularization term or using stochastic training (as in stochastic gradient descent) can jostle the search trajectory enough to escape shallow local minima [8]. Techniques like momentum or gradient noise can help the optimizer roll through small bumps in the loss landscape [9]. However, these are not guaranteed fixes, and truly complex landscapes may still confound purely local methods.

In summary, **global optimization** aims to **find the single best optimum among many** [10]. It requires strategies to explore multiple basins of attraction, whereas **local optimization** hones in on one basin and finds the optimum there [10]. We will now focus on one versatile global strategy – **Basin Hopping** – which essentially performs *iterative random restarts of a local optimizer in a clever way* to search for the global minimum.

The Basin Hopping Algorithm for Global Optimization

Overview: *Basin Hopping* (also called **basin-hopping** or **Monte Carlo Minimization**) is a **two-phase global optimization method** that combines a random perturbation strategy with local optimization at each step [11]. It was originally developed in the late 1990s by David J. Wales and Jonathan Doye to tackle difficult chemical physics problems (finding the lowest-energy structures of atomic clusters) [12]. In fact, one 2004 study calls basin-hopping "one of the most reliable algorithms in chemical physics to search for the lowest-energy structure" [13]. But beyond physics, basin hopping is a general-purpose **metaheuristic** for nonlinear functions with many optima [14]. The good news is that it's conceptually simple and **straightforward to implement**, which makes it a great tool for practitioners and easy to combine with existing local optimization code.

Key Idea: Basin hopping transforms a rugged landscape into a series of *basins* (valleys), and then **explores those basins by hopping between them** [15]. The algorithm repeatedly perturbs the current solution randomly to jump to a new region, then performs a local minimization to find the bottom of the basin in that region, and decides whether to accept the new solution or not based on its energy (objective value). By doing this iteratively, it "hops" from basin to basin, hopefully eventually landing in the deepest basin (the global minimum). Essentially, it's like a kangaroo hopping around a landscape: it jumps to a random location, then quickly bounds down to the nearest valley floor; if that valley is deeper (lower) than where it came from, it likely stays there, otherwise it might hop away to try another valley.

Algorithm Steps: Each iteration (or "hop") of the basin hopping procedure consists of three main steps [11]:

1. **Random Perturbation (Step 1):** Randomly *perturb* the current solution x by a small amount to get a new starting point x_{new}. This is typically done by adding a random displacement to each coordinate: $x_{\text{new}} = x + \Delta$, where Δ is a random vector (often drawn uniformly from a range or with some step size). The perturbation is meant to jump out of the current basin of attraction and into a new region of the search space (hence "hopping"). The magnitude of this random step is a key parameter (often called `stepsize` in implementations) – it should be large enough to explore different basins, but not so large as to always jump to far-off irrelevant regions [16].

2. **Local Optimization (Step 2):** Take the perturbed point x_{new} and perform a **local minimization** (using any suitable local optimizer) to find the nearest local optimum in that region. In other words, "roll down to the bottom of the basin" starting from x_{new}. This yields a candidate solution x_{cand} which is the local minimum of the basin where x_{new} landed. The objective function value at this candidate, $f_{\text{cand}} = f(x_{\text{cand}})$, is the best found in that basin.

3. **Acceptance Criterion (Step 3):** Decide whether to **accept or reject** the candidate x_{cand} as the new current solution for the next iteration. The simplest rule is: if $f_{\text{cand}} < f_{\text{current}}$ (the candidate is lower/better than our current solution), accept it unconditionally [17]. If the candidate is higher (worse), the algorithm may still accept it with some probability, to allow escape from local minima. Basin hopping typically uses a **Metropolis acceptance criterion** from Monte Carlo methods [11] [18]: we accept a worse move with probability

$$P(\text{accept}) = \exp\Big(\frac{-(f_{\text{cand}} - f_{\text{current}})}{T}\Big),$$

   where T is a "temperature" parameter [17]. A higher temperature makes it more likely to accept uphill moves (similar to simulated annealing), while $T = 0$ would enforce strict downhill-only moves (this special case is sometimes called *Monotonic Basin Hopping* where no uphill steps are allowed) [17] [19]. The use of a temperature-based acceptance gives the algorithm a chance to climb out of a shallow basin by temporarily going to a higher state in hopes of finding a deeper basin beyond. In practice, early in the search one might use a relatively high temperature to allow more exploration, and then lower the temperature as the iterations progress to focus in on the best area (analogous to the cooling schedule in simulated annealing) [20].

After this accept/reject step, if the move was accepted, the algorithm sets the current solution $x := x_{\text{cand}}$ (hopping into that basin). If rejected, the algorithm stays at the current solution (effectively *bouncing off* the hillside and not entering that new basin). Then it iterates again with another random perturbation from the current position.

These steps repeat for a predetermined number of iterations or until some stopping criterion is met (for example, no improvement after many tries, or a time limit, etc.). The lowest solution encountered over all iterations is kept as the final answer [21]. Importantly, because this is a stochastic algorithm, there's no absolute guarantee that the true global minimum is found on a single run. In practice, one might run the whole basin hopping process multiple times from different initial points to increase confidence, or until the best result doesn't improve further [21].

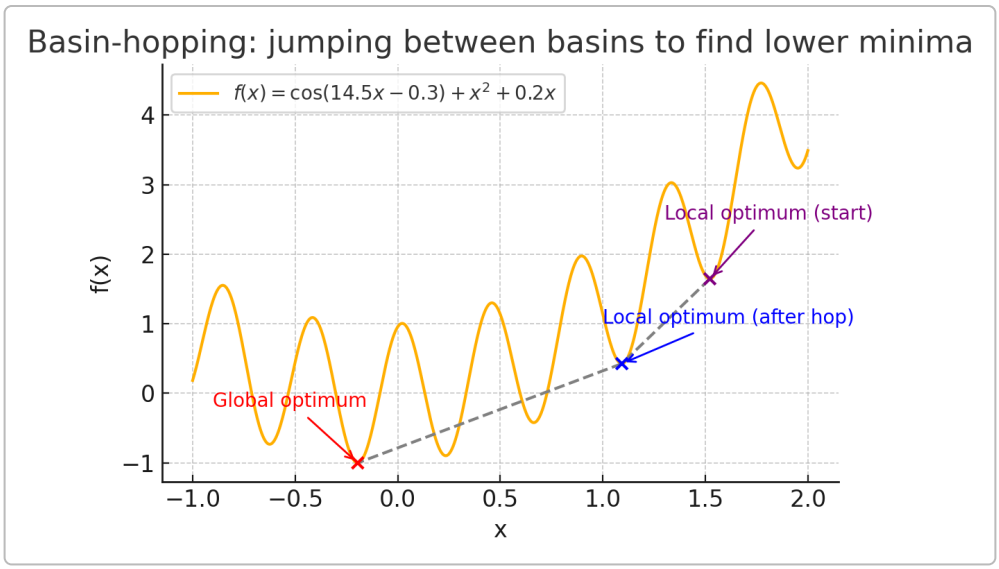

Figure: Illustration of the Basin Hopping process on the same example function. Starting from a local optimum in one basin (purple X at ~$x = 1.52$), a random hop (dashed gray arrow) takes the solution to a new region where a local search finds a lower local optimum (blue X at ~$x = 1.09$). The algorithm "hops" to that basin since it's an improvement. From there, another random perturbation lands the solution near the deepest valley, and a local search finds the global optimum (red X at $x \approx -0.195$). Basin hopping thus sequentially explores lower and lower basins.

In the figure above, the basin-hopping algorithm was able to jump out of the initial basin and eventually reach the global minimum by accepting moves that led to progressively lower minima. Note that sometimes the algorithm might accept a worse solution (an "uphill" move) if the temperature and random chance allow – for example, it might have temporarily moved to a slightly higher valley before finding an even lower one. This property of sometimes moving *up* before going further *down* is crucial; it's what allows basin hopping to escape traps that a purely greedy local search would not. By tuning the step size and acceptance temperature, one can balance exploration and exploitation: **exploration** (random jumps to diverse areas) vs. **exploitation** (locally optimizing a promising region) [22] .

When is Basin Hopping Effective? Basin hopping works especially well for problems that have a "rugged but funnel-like" structure [23] . This means there are many local minima but they tend to funnel down towards a dominant global minimum. Many physical energy landscapes have this property (lots of small wiggles but a deep ground state). The algorithm is known to be extremely efficient for a wide variety of physics and chemistry optimization problems [22] , and it has also proven competitive on standard mathematical test functions. Recent analyses have shown that Basin Hopping's performance can be on par with more complex metaheuristics like CMA-ES or differential evolution for many global optimization benchmarks [24] . Its appeal lies in its simplicity: it leverages any fast local solver and adds just enough stochastic perturbation to climb out of local optima.

Relationship to Other Methods: Basin hopping is conceptually related to *simulated annealing*, but instead of wandering via many small random steps (with occasional uphill moves), it takes *larger hops followed by full local minimizations*. In other words, simulated annealing performs a random walk on the actual function surface, while basin hopping performs a random walk on the *transformed landscape of basin minima*. Each basin hop effectively treats an entire basin as one "state." This can be more efficient in high dimensions

because it skips quickly to basin bottoms rather than slowly sliding down slopes. Basin hopping is also akin to a multi-start strategy with a clever accept/reject rule: it's like doing repeated local optimizations from random start points, but each new start point is informed by the last position and acceptance criterion. In practice, basin hopping can sometimes revisit the same basin multiple times (if hops are too small or if it accepts an uphill move out and then returns), but the hope is it will explore widely given enough iterations and a well-chosen step size.

Using Basin Hopping on Top of Local Optimizers

One great advantage of basin hopping is that it's **modular** – you can **pair it with any local optimization algorithm** of your choice. If you have developed or prefer a certain local optimizer (be it gradient descent, a quasi-Newton method, Nelder-Mead simplex, or a domain-specific heuristic), basin hopping can serve as an outer loop to amplify it into a global search method. The concept is straightforward:

- **Your local optimizer =** a procedure `LocalOptimize(x0)` that takes an initial guess and returns a refined solution at a nearby (local) minimum.
- **Basin hopping =** a wrapper around `LocalOptimize` that adds random jumps and an acceptance test.

In pseudo-code, a basic **basin hopping wrapper** could look like:

```
best = LocalOptimize(x0)           // Start by local-optimizing the initial
guess
for iteration = 1 to N:
    # Step 1: Random perturbation
    x_random = best + RandomStep()  // jump to a new point near the current best

    # Step 2: Local optimization in the new region
    candidate = LocalOptimize(x_random)

    # Step 3: Acceptance test (Metropolis criterion)
    Δf = f(candidate) - f(best)
    if Δf <= 0:
        best = candidate           // if candidate is better, accept it
    else:
        if rand() < exp(-Δf / T):
            best = candidate       // else accept with some probability (uphill
move)
        else:
            // reject: do nothing, best remains unchanged
```

You would also keep track of the lowest `f(best)` seen so far as the algorithm runs. Over iterations, `best` will (hopefully) move into deeper and deeper basins. The random step (`RandomStep()`) should be tuned to the problem's scale – too small and you only explore the current vicinity, too large and you might jump far from good regions. In many implementations, the step size is adjusted adaptively to achieve a target acceptance rate (commonly about 50%) [16] [25].

Incorporating Your Own Local Solver: If you have a custom local optimization routine, you can integrate it as shown above. Essentially, basin hopping will call your solver repeatedly on various starting points. The only "communication" needed is that your solver returns a solution x_{cand} and its function value f_{cand} . The basin hopping logic uses those to decide acceptance. You can thus **separate concerns**: treat your local optimizer as a black box that efficiently finds a nearby minimum, while the basin hopping loop handles the global exploration logic.

Using SciPy's Basin Hopping (with Custom Local Methods): The SciPy library provides a convenient ready-made implementation: `scipy.optimize.basinhopping`. You can use it to perform basin hopping on any objective function. It internally uses SciPy's `minimize` function for the local search, and you can **choose which local optimizer** to use by passing `minimizer_kwargs`. For example, if you want to use the BFGS algorithm for local minimization:

```
from scipy.optimize import basinhopping

def objective(x):
    return ...  # your objective function

x0 = [initial_guess]
res = basinhopping(objective, x0, niter=200, minimizer_kwargs={"method":
"BFGS"})
print(res.x, res.fun)  # prints the solution and objective value found
```

In this snippet, SciPy will perform 200 basin-hopping iterations, each time using the BFGS method to locally minimize the objective [26]. You can swap in any method available in `scipy.optimize.minimize` (Nelder-Mead, L-BFGS-B, CG, etc.) or even specify Jacobians if available [27] [28]. SciPy's implementation is highly customizable: you can plug in your own step-taking function or acceptance condition if needed [29] [30]. By default it uses uniform random displacements for steps and the Metropolis criterion for acceptance, as described earlier [11] [17]. It also will adjust the step size to try to maintain a good acceptance rate (unless you fix `stepsize` manually) [16]. If you have already written a custom local optimizer outside of SciPy's framework, you could still use SciPy's basin hopping by wrapping your optimizer as a custom `minimize` method or by writing a simple loop as in the pseudo-code above.

Tip: When using basin hopping (or any global method), it's wise to run it multiple times and/or with different random seeds, since it's stochastic. If all runs keep finding the same solution, that's a good indication it's the true global optimum. SciPy's result object for `basinhopping` will give you the best solution found, but it's up to you to be confident it's global – you might increase `niter` (number of hops) or try different initial guesses to be sure [21].

Comparison to Other Global Methods: Basin hopping is often a **great default choice** for global optimization on smooth functions because it's easy to use and often very effective [24]. Unlike genetic algorithms or particle swarm optimizers, it doesn't maintain a large population of points – it's basically doing a *smart random walk* guided by local minimization. This can make it more efficient in terms of function evaluations, since each hop uses gradient information (if available) to quickly find a nearby minimum. However, population-based methods might explore a space more diversely and can sometimes handle multi-modal surfaces with many distant optima better. Simulated annealing is somewhat similar in spirit,

but basin hopping's use of an actual local solver can give it a speed advantage on problems where gradients or efficient local methods exist. One can even hybridize strategies – e.g., run a population-based search to identify promising regions, then fine-tune with basin hopping or local searches. In practice, if you have a reasonably smooth objective and can compute it fairly quickly, basin hopping is a solid approach to try first for global optimization.

Conclusion

Local vs Global Recap: Local optimization is like finding the bottom of the nearest valley, whereas global optimization is about finding the lowest valley in the entire landscape [10]. Newcomers often inadvertently settle for the former when the problem actually demands the latter. Recognizing when a problem is multimodal or non-convex (hence prone to local minima) is crucial. In such cases, augmenting your optimization strategy to incorporate global search techniques is necessary to avoid suboptimal solutions.

Basin Hopping Highlights: The basin hopping algorithm provides a **conceptually simple yet powerful** way to augment any local optimizer with global search capability. It works by alternately exploring (random jumps) and exploiting (local minimization) the search space, analogous to having "multiple shots" at finding better optima while not wasting time wandering aimlessly. We saw how it can escape local minima traps by occasionally accepting uphill moves, all controlled by a temperature parameter akin to simulated annealing. The method has strong empirical track records in fields like chemistry, but it's equally applicable to engineering design problems, machine learning hyperparameter tuning, or any scenario with a tricky objective landscape.

Practical Advice: To effectively use basin hopping (or any global method), one should experiment with parameters like step size and temperature. If the algorithm isn't finding better solutions, consider increasing the step size to explore farther, or raising the temperature to allow more uphill moves. Conversely, if it's jumping too wildly, reduce the step size. Also remember that global optimization is inherently more computationally intensive than local optimization – you're doing many local solves, so be mindful of the cost per local optimization. SciPy's implementation makes it easy to try basin hopping; you can often turn a local solve into a global one with just a few extra lines of code. And if you've crafted a specialized local solver for your problem, wrapping it in a basin hopping loop could be a relatively quick way to improve its robustness against local minima.

In summary, **don't let local minima lull you into a false sense of security**. By understanding the difference between local and global optima and using tools like basin hopping, you can significantly increase the chances of finding the true best solution to your optimization problem. This not only helps avoid pitfalls in projects but also deepens your intuition about how complex optimization landscapes behave – a valuable insight for any data scientist or engineer.

Sources: The concepts and algorithms discussed here are grounded in optimization theory and have been referenced from a variety of resources, including MathWorks documentation [31] [4], academic papers on basin hopping [15] [11], and practical guides on global optimization [10] [17]. The SciPy library's documentation was referenced for details on their `basinhopping` function and usage examples [26]. These provide further reading for those interested in the technical underpinnings and variations of the basin hopping algorithm. Happy optimizing!

---

1 4 6 7 31 Local vs. Global Optima - MATLAB & Simulink
https://www.mathworks.com/help/optim/ug/local-vs-global-optima.html

2 5 8 9 The Curse of Local Minima: How to Escape and Find the Global Minimum | by Mohit Mishra | Medium
https://mohitmishra786687.medium.com/the-curse-of-local-minima-how-to-escape-and-find-the-global-minimum-fdabceb2cd6a

3 princeton.edu
https://www.princeton.edu/~aaa/Public/Teaching/ORF523/S16/ORF523_S16_Lec4_gh.pdf

10 12 13 14 15 20 Basin Hopping Optimization in Python - MachineLearningMastery.com
https://machinelearningmastery.com/basin-hopping-optimization-in-python/

11 16 17 18 19 21 22 23 25 26 27 28 29 30 basinhopping — SciPy v1.16.1 Manual
https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.basinhopping.html

24 A Performance Analysis of Basin Hopping Compared to Established Metaheuristics for Global Optimization
https://arxiv.org/html/2403.05877v1